

2520030408 – Bista Jamuna

SKILL 1

Aim:

To install and configure a Linux virtual machine, set up GCC and Git, create a project structure, understand basic shell architecture, and build an initial Makefile. The experiment also aims to study process abstraction by using `fork()` and the `exec()` family, observe parent-child relationships, inspect the process tree, and practice system call tracing.

Requirements:

Linux/Ubuntu Virtual Machine, GCC compiler, Git, Terminal, Make utility, and system-call tracing tools such as `strace`.

Procedure:

1. Install Ubuntu Linux in a virtual machine and open the terminal.
2. Update the package list and install GCC, Git, Make, and required development tools.

```

jamuna@Jem:~$ sudo apt update
[sudo: authenticate] Password:
Get:1 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Hit:2 http://archive.ubuntu.com/ubuntu resolute InRelease
Get:3 http://archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:4 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [417 kB]
Get:5 http://security.ubuntu.com/ubuntu resolute-security/main Translation-en [97.2 kB]
Get:6 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Components [46.7 kB]
Get:7 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Packages [159 kB]
Get:8 http://security.ubuntu.com/ubuntu resolute-security/universe Translation-en [51.2 kB]
Get:9 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Components [43.2 kB]
Get:10 http://security.ubuntu.com/ubuntu resolute-security/restricted amd64 Packages [349 kB]
Get:11 http://security.ubuntu.com/ubuntu resolute-security/restricted Translation-en [67.8 kB]
Get:12 http://security.ubuntu.com/ubuntu resolute-security/multiverse amd64 Packages [9020 B]
Get:13 http://security.ubuntu.com/ubuntu resolute-security/multiverse Translation-en [2660 B]
Get:14 http://archive.ubuntu.com/ubuntu resolute-backports InRelease [136 kB]
Get:15 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Packages [492 kB]
Get:16 http://archive.ubuntu.com/ubuntu resolute-updates/main Translation-en [118 kB]
Get:17 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Components [97.8 kB]
Get:18 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Packages [243 kB]
Get:19 http://archive.ubuntu.com/ubuntu resolute-updates/universe Translation-en [81.0 kB]
Get:20 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Components [184 kB]
Get:21 http://archive.ubuntu.com/ubuntu resolute-updates/restricted amd64 Packages [367 kB]
Get:22 http://archive.ubuntu.com/ubuntu resolute-updates/restricted Translation-en [70.8 kB]
Get:23 http://archive.ubuntu.com/ubuntu resolute-updates/multiverse amd64 Packages [11.3 kB]
Get:24 http://archive.ubuntu.com/ubuntu resolute-updates/multiverse Translation-en [3032 B]
Fetched 3320 kB in 4s (872 kB/s)
78 packages can be upgraded. Run 'apt list --upgradable' to see them.
jamuna@Jem:~$ sudo apt install gcc git make build-essential psmisc strace -y
gcc is already the newest version (4:15.2.0-5ubuntu1).
gcc set to manually installed.
git is already the newest version (1:2.53.0-1ubuntu1).
git set to manually installed.
make is already the newest version (4.4.1-3).
make set to manually installed.
build-essential is already the newest version (12.12ubuntu2.26.04.2).
psmisc is already the newest version (23.7-2ubuntu2).
psmisc set to manually installed.
strace is already the newest version (6.19+ds-0ubuntu5).
Summary:

```

3. Verify the installation using commands such as `gcc --version`, `git --version`, and `make --version`.

```

jamuna@Jem:~$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

jamuna@Jem:~$ git --version
git version 2.53.0
jamuna@Jem:~$ make --version
GNU Make 4.4.1
Built for x86_64-pc-linux-gnu
Copyright (C) 1988-2023 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <https://gnu.org/licenses/gpl.html>
This is free software; you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
jamuna@Jem:~$ strace -V
strace -- version 6.19
Copyright (c) 1991-2026 The strace developers <https://strace.io>.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

Optional features enabled: stack-trace=libunwind stack-demangle m32-mpers mx32-mpers

```

4. Configure Git using the user's name and email address.

```
jamuna@Jem:~$ git config --global user.name "Jamuna"
jamuna@Jem:~$ git config --global user.email "bistajamuna85@gmail.com"
jamuna@Jem:~$ git config --global --list
user.name=Jamuna
user.email=bistajamuna85@gmail.com
```

5. Create a project directory with separate folders for source files, header files, and build files.

```
jamuna@Jem:~$ mkdir -p OSSP_Skill/src OSSP_Skill/include OSSP_Skill/build
jamuna@Jem:~$ cd OSSP_Skill
jamuna@Jem:~/OSSP_Skill$ ls
build include src
```

6. Initialize the project as a Git repository using `git init`.
7. Create a simple C program and compile it using GCC.

```
jamuna@Jem:~$ nano hello.c
jamuna@Jem:~$ gcc hello.c -o hello
jamuna@Jem:~$ ./hello
Hello from Linux!
```

8. Create a basic Makefile to automate compilation and build the program using the `make` command.
9. Study the shell architecture by understanding how the shell accepts a command, creates a process, executes the requested program, and waits for its completion.

```
jamuna@Jem:~$ ls
Makefile      b1          execcv.c      f1          fork2.c      newfile1     pipex.c      readwrite.c
OSSP_Skill    b2          execcv.c.save f1.c        fork2.c.save open2.c       pipex2.c     sample.txt
a             c2          execl.c       f2          fork3.c       open2.c.save practical1.c  student_portal
a.c           content.c   execll.c      f3          fork3.c.save  pen2.c       practical2.c  tt
a7            d          execlp.c      f4          hello         permission_lab practical3.c
a8            dir5       execlp1.c     fi          hello.c       pipe.c       practicl2.c
ac            dir6       execlp1.c     fi          hello.c       pipe.c       practicl2.c
jamuna@Jem:~$ echo $$
3077
```

10. Write a C program using `fork()` to create a child process and observe the process IDs of both the parent and child.
11. Use the `exec()` family of system calls to replace the child process with another program or command.
12. Use commands such as `ps`, `pstree`, and `/proc` to inspect the parent-child relationship and process tree.
13. Use `strace` to trace the system calls made by a program and observe calls such as `fork()`, `execve()`, `wait()`, `open()`, and `read()`.

Linux environment was successfully installed and configured with GCC, Git, and Make. A project structure and Git repository were created successfully. The shell was observed to act as an interface between the user and the operating system by creating and managing processes for entered commands. The `fork()` system call created a new child process, while `exec()` replaced the child process with the specified program. The parent and child processes had different process IDs and their relationship could be observed using `ps` and `pstree`. System-call tracing using `strace` displayed the system calls performed during program execution.

The Linux development environment was successfully configured, a Git-based project structure and Makefile were created, and the concepts of process creation, process replacement, parent-child relationships, process-tree inspection, and system-call tracing were successfully demonstrated.